

Panduan Lengkap

Belajar Bahasa Pemrograman Go

Dari Dasar hingga Concurrency

Buku ini disusun secara orisinal dalam Bahasa Indonesia untuk membantu siapa saja mempelajari bahasa pemrograman Go (Golang) dari nol, mencakup sintaks dasar, struktur kontrol, fungsi, struktur data, method & interface, hingga model concurrency Go (goroutine, channel, select). Setiap bab dilengkapi penjelasan konsep dan contoh kode yang bisa langsung dicoba.

Daftar Isi

BAGIAN 1 — MEMULAI DENGAN GO

1. Apa itu Go?
2. Instalasi Go
3. Program Pertama: Hello, World!

BAGIAN 2 — DASAR-DASAR GO

4. Variabel
5. Konstanta
6. String

BAGIAN 3 — ALUR KONTROL (CONTROL FLOW)

7. Pernyataan If...Else
8. Pernyataan Switch
9. Perulangan For
10. Perulangan For...Range
11. Break
12. Continue

BAGIAN 4 — FUNGSI

13. Fungsi (Functions)
14. Fungsi dengan Banyak Nilai Kembali
15. Fungsi sebagai First-Class Citizen
16. Defer
17. Pointer dan Pass by Value

BAGIAN 5 — STRUKTUR DATA

18. Array
19. Slice
20. Map
21. Struct

BAGIAN 6 — METHOD DAN INTERFACE

22. Method
23. Interface
24. Empty Interface (interface{})

BAGIAN 7 — CONCURRENCY

25. Goroutine
26. Channel
27. Select

BAGIAN 8 — ORGANISASI KODE & PERKAKAS

28. Package
29. Live Reload untuk Pengembangan

Memulai dengan Go

1. Apa itu Go?

Go (sering disebut **Golang**) adalah bahasa pemrograman open-source yang dikembangkan oleh Google, pertama kali diperkenalkan pada tahun 2009 oleh Robert Griesemer, Rob Pike, dan Ken Thompson. Go dirancang untuk mengatasi masalah yang sering dihadapi dalam pengembangan perangkat lunak berskala besar: waktu kompilasi yang lambat, ketergantungan yang rumit, dan kesulitan menulis program yang berjalan secara paralel.

Ciri-ciri utama Go

- **Statically typed & compiled** — tipe data diperiksa saat kompilasi, dan program dikompilasi langsung menjadi kode mesin (binary) sehingga berjalan sangat cepat, tanpa perlu interpreter atau virtual machine terpisah saat runtime.
- **Sintaks sederhana** — Go sengaja dirancang minimalis; jumlah kata kunci (keyword) jauh lebih sedikit dibanding bahasa seperti Java atau C++, sehingga lebih cepat dipelajari.
- **Garbage collected** — manajemen memori otomatis, sehingga programmer tidak perlu mengalokasikan dan membebaskan memori secara manual seperti di C/C++.
- **Concurrency sebagai fitur inti** — Go menyediakan goroutine dan channel langsung sebagai bagian dari bahasa, membuat penulisan program konkuren jauh lebih mudah dibanding thread tradisional.
- **Kompilasi sangat cepat** dan menghasilkan satu file binary mandiri (statically linked) yang mudah didistribusikan tanpa perlu menginstal dependency tambahan di server tujuan.

Go banyak digunakan untuk membangun backend layanan web, microservices, command-line tools (CLI), sistem infrastruktur cloud (Docker dan Kubernetes ditulis dalam Go), hingga sistem jaringan berskala besar.

2. Instalasi Go

Sebelum menulis kode Go, kita perlu memasang Go SDK (Software Development Kit) di komputer.

Langkah instalasi (Windows, macOS, Linux)

- Kunjungi situs resmi <https://go.dev/dl/> dan unduh installer sesuai sistem operasi (Windows: .msi, macOS: .pkg, Linux: .tar.gz).
- **Windows:** jalankan file .msi yang telah diunduh, ikuti wizard instalasi, lalu restart terminal/command prompt.
- **macOS:** jalankan file .pkg lalu ikuti petunjuk instalasi grafis, atau gunakan Homebrew dengan perintah `brew install go`.
- **Linux:** ekstrak arsip .tar.gz ke direktori `/usr/local`, lalu tambahkan `/usr/local/go/bin` ke variabel lingkungan PATH pada berkas `~/.bashrc` atau `~/.zshrc`.

Setelah instalasi selesai, verifikasi dengan menjalankan perintah berikut di terminal:

```
go version
```

Output:

```
go version go1.23.0 darwin/amd64
```

Jika versi Go muncul seperti contoh di atas, instalasi berhasil dan siap digunakan.

3. Program Pertama: Hello, World!

Seperti tradisi dalam belajar bahasa pemrograman apa pun, program pertama yang kita buat adalah program sederhana yang menampilkan tulisan 'Hello, World!' ke layar.

Langkah-langkah

- Buat folder proyek baru, misalnya `hello-go`, lalu masuk ke dalamnya melalui terminal.
- Inisialisasi modul Go dengan perintah `go mod init hello`. Perintah ini membuat berkas `go.mod` yang mendefinisikan nama modul dan versi Go yang dipakai.
- Buat berkas baru bernama `main.go` lalu isi dengan kode berikut:

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

Jalankan program dengan perintah:

```
go run main.go
```

Output:

```
Hello, World!
```

Penjelasan kode

- `package main` — setiap berkas Go harus diawali deklarasi package. Package bernama `main` menandakan bahwa berkas ini adalah titik masuk (entry point) program yang bisa dikompilasi menjadi executable, bukan sekadar pustaka (library).
- `import "fmt"` — mengimpor package standar `fmt` (format) yang berisi fungsi-fungsi untuk mencetak teks ke layar dan memformat string.
- `func main()` — fungsi `main` adalah fungsi yang otomatis dijalankan pertama kali saat program dieksekusi.
- `fmt.Println(...)` — memanggil fungsi `Println` dari package `fmt` untuk mencetak teks diikuti baris baru.

Selain `go run` (menjalankan langsung tanpa menyimpan file binary), kita juga bisa mengompilasi program menjadi file executable mandiri dengan perintah `go build main.go`, yang akan menghasilkan file biner yang bisa dijalankan tanpa perlu Go terinstal di komputer lain.

Dasar-Dasar Go

4. Variabel

Variabel adalah tempat penyimpanan data yang bisa diberi nama dan nilainya dapat berubah selama program berjalan. Go menyediakan beberapa cara untuk mendeklarasikan variabel.

Deklarasi menggunakan kata kunci var

```
var nama string = "Andi"
var umur int = 25
var tinggiBadan float64 = 172.5
```

Jika nilai awal langsung diberikan, Go bisa menebak (infer) tipe datanya secara otomatis, sehingga tipe data boleh dihilangkan:

```
var nama = "Andi"      // otomatis bertipe string
var umur = 25          // otomatis bertipe int
```

Deklarasi singkat dengan := (short variable declaration)

Di dalam fungsi, cara paling umum untuk mendeklarasikan variabel adalah menggunakan operator :=, yang otomatis membuat variabel baru sekaligus menentukan tipenya dari nilai di sebelah kanan:

```
func main() {
    nama := "Budi"
    umur := 30
    fmt.Println(nama, umur)
}
```

Operator := hanya bisa digunakan di dalam fungsi, bukan di level package (global). Untuk variabel global, wajib menggunakan kata kunci var.

Deklarasi banyak variabel sekaligus

```
var (
    nama    string = "Citra"
    umur    int    = 22
    aktif   bool   = true
)
```

Zero value

Jika sebuah variabel dideklarasikan tanpa nilai awal, Go otomatis memberinya nilai default sesuai tipe datanya, yang disebut *zero value*: 0 untuk tipe numerik, "" (string kosong) untuk string, dan false untuk boolean.

```
var jumlah int      // bernilai 0
var pesan string    // bernilai ""
var selesai bool    // bernilai false
```

5. Konstanta

Konstanta adalah nilai yang ditetapkan sekali dan tidak bisa diubah selama program berjalan. Konstanta dideklarasikan menggunakan kata kunci `const`.

```
const Pi = 3.14159
const NamaAplikasi = "Toko Online"

func main() {
    fmt.Println(Pi, NamaAplikasi)
}
```

Berbeda dengan variabel, nilai konstanta harus sudah diketahui saat proses kompilasi (bukan hasil pemanggilan fungsi saat runtime), dan mencoba mengubah nilainya akan menghasilkan error kompilasi.

Menggunakan `iota` untuk konstanta berurutan

Go menyediakan identifier khusus bernama `iota` yang berguna untuk membuat rangkaian konstanta bertipe enum secara ringkas:

```
type HariKerja int

const (
    Senin HariKerja = iota // 0
    Selasa              // 1
    Rabu                // 2
    Kamis               // 3
    Jumat               // 4
)
```

Setiap baris baru dalam blok `const` yang menggunakan `iota` otomatis menambah nilainya sebesar 1, dimulai dari 0 pada baris pertama.

6. String

String di Go merepresentasikan rangkaian karakter (teks) dan bersifat **immutable**, artinya setelah dibuat, isinya tidak bisa diubah langsung — operasi seperti penggabungan sebenarnya membuat string baru, bukan mengubah string lama.

```
nama := "Go"
sapaan := "Halo, " + nama + "!"
fmt.Println(sapaan)
```

Output:
Halo, Go!

Fungsi umum pada package `strings`

- `strings.ToUpper(s)` — mengubah semua huruf menjadi kapital.
- `strings.ToLower(s)` — mengubah semua huruf menjadi huruf kecil.
- `strings.Contains(s, substr)` — memeriksa apakah suatu string mengandung substring tertentu.
- `strings.Split(s, sep)` — memecah string menjadi slice berdasarkan pemisah tertentu.
- `strings.TrimSpace(s)` — menghapus spasi di awal dan akhir string.
- `len(s)` — mengembalikan panjang string dalam satuan byte (bukan jumlah karakter Unicode).

```
import (  
    "fmt"  
    "strings"  
)  
  
func main() {  
    kalimat := "Belajar Bahasa Go"  
    fmt.Println(strings.ToUpper(kalimat))  
    fmt.Println(strings.Contains(kalimat, "Go"))  
    fmt.Println(len(kalimat))  
}
```

Output:

```
BELAJAR BAHASA GO  
true  
18
```

Karena string di Go disimpan sebagai urutan byte UTF-8, karakter non-ASCII (misalnya emoji atau huruf beraksen) bisa terdiri dari lebih dari satu byte. Jika ingin bekerja per karakter (rune), gunakan tipe rune atau perulangan `for range`.

Alur Kontrol (Control Flow)

7. Pernyataan If...Else

Pernyataan if digunakan untuk menjalankan blok kode tertentu hanya jika suatu kondisi bernilai benar (true).

```
umur := 20

if umur >= 18 {
    fmt.Println("Anda sudah dewasa")
} else if umur >= 13 {
    fmt.Println("Anda remaja")
} else {
    fmt.Println("Anda masih anak-anak")
}
```

Output:
Anda sudah dewasa

Keunikan Go: kondisi pada if tidak perlu diapit tanda kurung, tetapi kurung kurawal { } wajib ada meskipun isinya cuma satu baris.

Statement inisialisasi pada if

Go memperbolehkan menuliskan sebuah pernyataan singkat sebelum kondisi, dipisahkan titik koma. Variabel yang dideklarasikan di sana hanya berlaku di dalam blok if/else tersebut:

```
if nilai := hitungSkor(); nilai >= 60 {
    fmt.Println("Lulus dengan nilai", nilai)
} else {
    fmt.Println("Tidak lulus, nilai", nilai)
}
```

8. Pernyataan Switch

Switch digunakan untuk membandingkan satu nilai dengan beberapa kemungkinan nilai lain, sebagai alternatif yang lebih ringkas dari rangkaian if-else panjang.

```
hari := "Rabu"

switch hari {
case "Senin":
    fmt.Println("Awal minggu")
case "Sabtu", "Minggu":
    fmt.Println("Akhir pekan")
default:
    fmt.Println("Hari biasa")
}
```

Output:
Hari biasa

- Berbeda dengan bahasa seperti C atau Java, setiap case di Go otomatis berhenti setelah dieksekusi (tidak ada *fall-through* otomatis), sehingga tidak perlu menulis `break` di setiap case.
- Satu case boleh memuat beberapa nilai sekaligus, dipisahkan koma, seperti case `"Sabtu", "Minggu":`.
- Jika ingin perilaku *fall-through* (lanjut ke case berikutnya) secara eksplisit, gunakan kata kunci `fallthrough`.
- Switch juga bisa ditulis tanpa nilai pembanding (disebut *switch true*), yang berfungsi seperti rangkaian if-else:

```
nilai := 75

switch {
case nilai >= 90:
    fmt.Println("A")
case nilai >= 75:
    fmt.Println("B")
default:
    fmt.Println("C")
}
```

9. Perulangan For

Go hanya memiliki satu kata kunci untuk perulangan, yaitu `for`, tetapi bisa ditulis dalam beberapa bentuk berbeda sesuai kebutuhan.

Bentuk klasik (tiga komponen)

```
for i := 0; i < 5; i++ {
    fmt.Println("Perulangan ke-", i)
}
```

Bentuk seperti while

```
hitung := 0
for hitung < 3 {
    fmt.Println("Hitung:", hitung)
    hitung++
}
```

Perulangan tanpa henti (infinite loop)

```
for {
    fmt.Println("Berjalan terus sampai di-break")
    break
}
```

Karena Go tidak memiliki kata kunci `while` atau `do-while` terpisah, semua bentuk perulangan — termasuk perulangan tak terbatas — dinyatakan menggunakan `for` dengan jumlah komponen yang berbeda-beda.

10. Perulangan For...Range

Bentuk `for...range` digunakan khusus untuk mengiterasi elemen dari koleksi data seperti array, slice, map, atau string, tanpa perlu mengelola index secara manual.

```
buah := []string{"Apel", "Jeruk", "Mangga"}
```

```
for index, nilai := range buah {
    fmt.Println(index, nilai)
}
```

Output:

```
0 Apel
1 Jeruk
2 Mangga
```

Jika index tidak diperlukan, gunakan garis bawah `_` sebagai penanda 'nilai ini sengaja diabaikan':

```
for _, nilai := range buah {
    fmt.Println(nilai)
}
```

Pada map, `for...range` memberikan pasangan key dan value:

```
umur := map[string]int{"Andi": 25, "Budi": 30}

for nama, usia := range umur {
    fmt.Println(nama, "berumur", usia, "tahun")
}
```

11. Break

Pernyataan `break` digunakan untuk menghentikan perulangan secara langsung, tanpa menunggu kondisi perulangan menjadi salah.

```
for i := 0; i < 10; i++ {
    if i == 4 {
        break
    }
    fmt.Println(i)
}
```

Output:

```
0
1
2
3
```

Untuk perulangan bersarang (nested loop), `break` secara default hanya menghentikan loop terdalam. Jika ingin menghentikan loop luar, gunakan *label*:

```
luar:
for i := 0; i < 3; i++ {
    for j := 0; j < 3; j++ {
        if j == 1 {
            break luar
        }
        fmt.Println(i, j)
    }
}
```

12. Continue

Pernyataan `continue` digunakan untuk melompati sisa kode dalam iterasi saat ini dan langsung melanjutkan ke iterasi berikutnya.

```
for i := 0; i < 6; i++ {  
    if i%2 == 0 {  
        continue // lompat angka genap  
    }  
    fmt.Println(i)  
}
```

Output:

1
3
5

Fungsi

13. Fungsi (Functions)

Fungsi adalah blok kode yang mengelompokkan logika tertentu agar bisa dipanggil ulang kapan saja tanpa menulis ulang kodenya. Fungsi dideklarasikan menggunakan kata kunci `func`.

```
func tambah(a int, b int) int {  
    return a + b  
}  
  
func main() {  
    hasil := tambah(5, 3)  
    fmt.Println(hasil)  
}
```

Output:
8

Jika beberapa parameter berurutan memiliki tipe yang sama, tipe data cukup ditulis sekali di parameter terakhir: `func tambah(a, b int) int`.

14. Fungsi dengan Banyak Nilai Kembali

Salah satu keunikan Go adalah kemampuan sebuah fungsi mengembalikan lebih dari satu nilai sekaligus, yang sangat umum dipakai untuk mengembalikan hasil sekaligus informasi error.

```
func bagi(a, b float64) (float64, error) {  
    if b == 0 {  
        return 0, fmt.Errorf("tidak bisa membagi dengan nol")  
    }  
    return a / b, nil  
}  
  
func main() {  
    hasil, err := bagi(10, 2)  
    if err != nil {  
        fmt.Println("Error:", err)  
        return  
    }  
    fmt.Println("Hasil:", hasil)  
}
```

Output:
Hasil: 5

Pola nilai, `err := fungsi()` lalu memeriksa `if err != nil` adalah idiom penanganan error yang sangat khas dan konsisten dipakai di hampir seluruh ekosistem Go.

15. Fungsi sebagai First-Class Citizen

Di Go, fungsi diperlakukan sebagai nilai (value) seperti tipe data lain. Artinya, fungsi bisa disimpan dalam variabel, dikirim sebagai argumen ke fungsi lain, maupun dikembalikan sebagai hasil dari sebuah fungsi.

Menyimpan fungsi ke dalam variabel

```
kali := func(a, b int) int {  
    return a * b  
}  
  
fmt.Println(kali(4, 5))
```

Output:
20

Mengirim fungsi sebagai argumen (higher-order function)

```
func operasi(a, b int, f func(int, int) int) int {  
    return f(a, b)  
}  
  
func main() {  
    hasil := operasi(6, 2, func(a, b int) int {  
        return a - b  
    })  
    fmt.Println(hasil)  
}
```

Output:
4

Mengembalikan fungsi dari fungsi lain (closure)

```
func buatPengali(faktor int) func(int) int {  
    return func(n int) int {  
        return n * faktor  
    }  
}  
  
func main() {  
    kaliTiga := buatPengali(3)  
    fmt.Println(kaliTiga(7))  
}
```

Output:
21

Fungsi yang dikembalikan pada contoh di atas disebut *closure* karena ia 'mengingat' nilai variabel faktor dari lingkup tempat ia dibuat, meskipun fungsi luarnya sudah selesai dieksekusi.

16. Defer Functions

Kata kunci defer digunakan untuk menunda eksekusi sebuah pemanggilan fungsi hingga fungsi yang membungkusnya (fungsi induk) selesai dijalankan — baik selesai secara normal maupun karena panic.

```
func bacaFile() {  
    fmt.Println("Membuka file")  
    defer fmt.Println("Menutup file")  
    fmt.Println("Membaca isi file")  
}
```

```
}
```

Output:

```
Membuka file  
Membaca isi file  
Menutup file
```

Pola ini sangat berguna untuk memastikan proses pembersihan (cleanup) — seperti menutup file, koneksi database, atau melepas lock — selalu dijalankan, tidak peduli jalur eksekusi mana yang diambil fungsi tersebut.

- Jika ada beberapa defer dalam satu fungsi, urutan eksekusinya adalah **LIFO** (Last In, First Out) — defer yang ditulis paling akhir akan dijalankan paling pertama.
- Argumen pada pemanggilan defer dievaluasi **saat baris defer dijalankan**, bukan saat fungsi benar-benar dipanggil nanti.

17. Go Pointers dan Pass by Value

Secara default, Go menggunakan mekanisme **pass by value**: ketika sebuah variabel dikirim sebagai argumen ke fungsi, yang dikirim adalah salinan nilainya, bukan variabel aslinya. Perubahan pada parameter di dalam fungsi tidak akan memengaruhi variabel asli di luar fungsi.

```
func tambahSatu(n int) {  
    n = n + 1  
}  
  
func main() {  
    angka := 5  
    tambahSatu(angka)  
    fmt.Println(angka) // tetap 5  
}
```

Output:

```
5
```

Untuk mengizinkan fungsi mengubah nilai asli, kita perlu mengirim **alamat memori** dari variabel tersebut menggunakan **pointer**, ditandai dengan operator **&** (mengambil alamat) dan ***** (mengakses/mengubah nilai yang ditunjuk).

```
func tambahSatu(n *int) {  
    *n = *n + 1  
}  
  
func main() {  
    angka := 5  
    tambahSatu(&angka)  
    fmt.Println(angka) // menjadi 6  
}
```

Output:

```
6
```

&variabel menghasilkan pointer yang menunjuk ke alamat variabel tersebut, sedangkan ***pointer** (dereference) digunakan untuk membaca atau mengubah nilai yang ditunjuk oleh pointer itu.

Struktur Data

18. Array

Array adalah kumpulan elemen dengan tipe data yang sama dan **ukuran tetap** (tidak bisa diperbesar atau diperkecil setelah dideklarasikan).

```
var angka [5]int
angka[0] = 10
angka[1] = 20
fmt.Println(angka)

buah := [3]string{"Apel", "Jeruk", "Mangga"}
fmt.Println(buah)
fmt.Println(len(buah)) // panjang array
```

Output:
 [10 20 0 0 0]
 [Apel Jeruk Mangga]
 3

Karena ukurannya tetap dan menjadi bagian dari tipe datanya sendiri (array berukuran 3 dan array berukuran 5 dianggap tipe berbeda), array jarang dipakai langsung di kode Go sehari-hari — sebagai gantinya, programmer Go jauh lebih sering menggunakan **slice**.

19. Slice

Slice adalah struktur data fleksibel yang dibangun di atas array, tetapi ukurannya dapat bertambah secara dinamis. Slice adalah struktur data koleksi paling umum dipakai dalam program Go.

```
buah := []string{"Apel", "Jeruk"}
buah = append(buah, "Mangga")
fmt.Println(buah)
fmt.Println(len(buah))
```

Output:
 [Apel Jeruk Mangga]
 3

Membuat slice dengan make

```
angka := make([]int, 0, 10) // panjang 0, kapasitas 10
for i := 1; i <= 5; i++ {
    angka = append(angka, i*i)
}
fmt.Println(angka)
```

Output:
 [1 4 9 16 25]

Slicing (mengambil sebagian slice)

```
angka := []int{10, 20, 30, 40, 50}
sebagian := angka[1:4] // index 1 sampai sebelum 4
```

```
fmt.Println(sebagian)
```

Output:

```
[20 30 40]
```

Sebuah slice sebenarnya hanyalah 'jendela pandang' (view) ke array di baliknya, terdiri dari pointer ke elemen awal, panjang (length), dan kapasitas (capacity). Ini penting dipahami karena dua slice bisa berbagi array yang sama di belakang layar, sehingga mengubah satu slice terkadang memengaruhi slice lain yang menunjuk ke array yang sama.

20. Map

Map adalah struktur data yang menyimpan pasangan **key-value**, mirip dictionary di Python atau object di JavaScript. Setiap key bersifat unik dan digunakan untuk mengakses value yang terkait.

```
umur := map[string]int{
    "Andi": 25,
    "Budi": 30,
}

fmt.Println(umur["Andi"])

umur["Citra"] = 22      // menambah pasangan baru
delete(umur, "Budi")    // menghapus pasangan

nilai, ada := umur["Budi"]
fmt.Println(nilai, ada) // 0 false, karena sudah dihapus
```

Output:

```
25
0 false
```

Pola `nilai, ada := map[key]` di atas disebut *comma-ok idiom*, berguna untuk membedakan antara 'key tidak ada' dengan 'key ada tapi valuenya kebetulan zero value'.

21. Struct

Struct adalah tipe data komposit yang mengelompokkan beberapa field (bidang) dengan tipe berbeda-beda menjadi satu kesatuan, mirip 'class tanpa method' pada bahasa berorientasi objek lain, atau seperti membuat model/entitas data sendiri.

```
type Pengguna struct {
    Nama  string
    Umur  int
    Email string
}

func main() {
    p := Pengguna{
        Nama:  "Dewi",
        Umur:  28,
        Email: "dewi@contoh.com",
    }
    fmt.Println(p.Nama, p.Umur)
}
```

Output:

```
Dewi 28
```

Struct bersarang (nested struct)

```
type Alamat struct {
    Kota    string
    KodePos string
}

type Pengguna struct {
    Nama    string
    Alamat Alamat
}

p := Pengguna{
    Nama: "Eka",
    Alamat: Alamat{Kota: "Bandung", KodePos: "40123"},
}
fmt.Println(p.Alatam.Kota)
```

Output:

Bandung

Method dan Interface

22. Method

Method adalah fungsi yang terikat pada suatu tipe tertentu (biasanya struct), didefinisikan dengan menambahkan sebuah *receiver* di antara kata kunci `func` dan nama method.

```
type Persegi struct {
    Sisi float64
}

func (p Persegi) Luas() float64 {
    return p.Sisi * p.Sisi
}

func main() {
    kotak := Persegi{Sisi: 4}
    fmt.Println(kotak.Luas())
}
```

Output:
16

Value receiver vs pointer receiver

Receiver bisa berupa **value receiver** (menerima salinan) atau **pointer receiver** (menerima alamat, sehingga bisa mengubah data aslinya). Pointer receiver digunakan ketika method perlu memodifikasi field struct aslinya:

```
func (p *Persegi) UbahSisi(sisiBaru float64) {
    p.Sisi = sisiBaru
}

func main() {
    kotak := Persegi{Sisi: 4}
    kotak.UbahSisi(10)
    fmt.Println(kotak.Sisi) // 10
}
```

Output:
10

23. Interface

Interface di Go mendefinisikan sekumpulan method yang harus dimiliki suatu tipe, tanpa peduli tipe apa itu sebenarnya. Sebuah tipe secara otomatis dianggap 'mengimplementasikan' interface jika ia memiliki seluruh method yang disyaratkan — tanpa perlu deklarasi eksplisit seperti `implements` di Java.

```
type Bentuk interface {
    Luas() float64
}
```

```

type Lingkaran struct {
    JariJari float64
}

func (l Lingkaran) Luas() float64 {
    return 3.14 * l.JariJari * l.JariJari
}

func cetakLuas(b Bentuk) {
    fmt.Println("Luas:", b.Luas())
}

func main() {
    lingkaran := Lingkaran{JariJari: 3}
    cetakLuas(lingkaran) // Persegi dari contoh sebelumnya juga bisa dikirim ke sin
i
}

```

Output:

Luas: 28.26

Konsep ini disebut *structural typing* atau sering dijuluki 'jika terlihat seperti bebek dan berjalan seperti bebek, maka itu bebek' (duck typing) — kecocokan tipe ditentukan dari perilaku (method) yang dimiliki, bukan dari relasi pewarisan (inheritance) eksplisit. Ini membuat kode Go sangat fleksibel karena fungsi seperti cetakLuas di atas bisa menerima tipe apa pun asalkan memiliki method Luas().

24. Empty Interface: interface{}

Interface kosong, ditulis interface{} (atau alias any sejak Go 1.18), tidak mensyaratkan method apa pun — sehingga bisa menampung nilai bertipe apa saja, mirip konsep Object di Java atau tipe any di TypeScript.

```

func cetakApapun(nilai interface{}) {
    fmt.Println(nilai)
}

func main() {
    cetakApapun(42)
    cetakApapun("halo")
    cetakApapun(true)
}

```

Output:

42
halo
true

Type assertion dan type switch

Karena nilai dalam empty interface bisa bertipe apa saja, kita sering perlu memeriksa tipe aslinya sebelum memprosesnya lebih lanjut, menggunakan *type assertion* atau *type switch*:

```

func deskripsikan(nilai interface{}) {
    switch v := nilai.(type) {
    case int:
        fmt.Println("Ini bilangan bulat:", v)
    case string:

```

```
    fmt.Println("Ini teks:", v)
default:
    fmt.Println("Tipe tidak dikenali")
}
```

Sejak Go 1.18, penggunaan `any` lebih disarankan sebagai alias yang lebih ringkas untuk `interface{}`, dan Go juga memperkenalkan **generics** sebagai alternatif yang lebih aman-tipe (type-safe) dibanding empty interface untuk banyak kasus.

Concurrency

25. Goroutine

Goroutine adalah unit eksekusi ringan yang dikelola langsung oleh runtime Go, memungkinkan sebuah fungsi berjalan secara konkuren (bersamaan) dengan fungsi lain. Goroutine jauh lebih ringan dibanding thread sistem operasi tradisional — sebuah program Go bisa menjalankan ribuan hingga jutaan goroutine sekaligus tanpa membebani sistem secara berlebihan.

```
func cetakPesan(pesan string) {
    for i := 0; i < 3; i++ {
        fmt.Println(pesan)
    }
}

func main() {
    go cetakPesan("berjalan konkuren") // dijalankan sebagai goroutine
    cetakPesan("berjalan normal")
}
```

Cukup menambahkan kata kunci `go` di depan pemanggilan fungsi untuk menjalankannya sebagai goroutine terpisah, berjalan bersamaan dengan goroutine lain (termasuk fungsi `main` itu sendiri).

Karena `main()` tidak menunggu goroutine lain selesai secara otomatis, program utama bisa saja berakhir sebelum goroutine yang di-`go`-kan sempat berjalan. Untuk menyinkronkan goroutine, biasanya digunakan `channel` atau `sync.WaitGroup` dari package standar `sync`.

26. Channel

Channel adalah 'saluran' yang digunakan goroutine untuk saling mengirim dan menerima data secara aman, tanpa perlu mekanisme locking manual. Filosofi Go terkenal dengan semboyan: *'Jangan berkomunikasi dengan berbagi memori; sebaliknya, berbagi memori dengan berkomunikasi.'*

```
func main() {
    ch := make(chan string)

    go func() {
        ch <- "data dari goroutine" // mengirim ke channel
    }()

    pesan := <-ch // menerima dari channel
    fmt.Println(pesan)
}
```

Output:
data dari goroutine

Secara default, channel bersifat **unbuffered**: pengirim akan menunggu (blocking) hingga ada penerima yang siap membaca, dan sebaliknya. Ini otomatis menyinkronkan dua goroutine tanpa lock manual.

Buffered channel

Channel juga bisa diberi kapasitas buffer, sehingga pengirim tidak langsung blocking selama buffer belum penuh:

```
ch := make(chan int, 3) // buffer berkapasitas 3
ch <- 1
ch <- 2
ch <- 3
close(ch) // menutup channel setelah selesai mengirim

for nilai := range ch {
    fmt.Println(nilai)
}
```

Output:

```
1
2
3
```

27. Select

Pernyataan `select` memungkinkan sebuah goroutine menunggu beberapa operasi channel sekaligus, dan akan mengeksekusi case dari channel mana pun yang lebih dulu siap — mirip `switch`, tetapi khusus untuk operasi channel.

```
func main() {
    ch1 := make(chan string)
    ch2 := make(chan string)

    go func() { ch1 <- "hasil dari ch1" }()
    go func() { ch2 <- "hasil dari ch2" }()

    for i := 0; i < 2; i++ {
        select {
            case pesan1 := <-ch1:
                fmt.Println(pesan1)
            case pesan2 := <-ch2:
                fmt.Println(pesan2)
        }
    }
}
```

Select sangat berguna untuk mengelola beberapa sumber data konkuren sekaligus, misalnya menunggu hasil dari beberapa permintaan API paralel, atau mengimplementasikan mekanisme timeout menggunakan kombinasi dengan package `time`:

```
select {
case hasil := <-ch:
    fmt.Println("Diterima:", hasil)
case <-time.After(2 * time.Second):
    fmt.Println("Timeout, tidak ada respons dalam 2 detik")
}
```

Organisasi Kode & Perkakas

28. Package

Package adalah cara Go mengorganisasikan kode ke dalam unit-unit yang bisa dipakai ulang. Setiap berkas Go harus menjadi bagian dari sebuah package, dinyatakan di baris pertama berkas.

- **package main** — package khusus yang menandakan program bisa dikompilasi menjadi executable, harus memiliki fungsi `main()`.
- **Package lain (non-main)** — berfungsi sebagai pustaka (library) yang bisa diimpor oleh package lain, misalnya package bawaan `fmt`, `strings`, atau `net/http`.
- **Kapitalisasi nama menentukan visibilitas:** identifier (fungsi, variabel, tipe) yang diawali huruf kapital bisa diakses dari package lain (exported/public), sedangkan yang diawali huruf kecil hanya bisa diakses dalam package yang sama (unexported/private).

```
// berkas: matematika/hitung.go
package matematika

func Tambah(a, b int) int { // exported, bisa dipakai package lain
    return a + b
}

func kurangiInternal(a, b int) int { // unexported, hanya untuk package ini
    return a - b
}

// berkas: main.go
package main

import (
    "fmt"
    "namamodul/matematika"
)

func main() {
    fmt.Println(matematika.Tambah(3, 4))
}
```

Go menggunakan sistem **module** (didefinisikan lewat berkas `go.mod`) untuk mengelola dependency proyek, termasuk pustaka pihak ketiga yang diinstal lewat perintah `go get`.

29. Live Reload untuk Pengembangan

Secara default, setiap kali kode Go diubah, kita perlu menghentikan program dan menjalankan ulang `go run` secara manual. Untuk mempercepat alur kerja saat development, banyak developer Go menggunakan tool **live reload** yang otomatis mengompilasi ulang dan menjalankan ulang program setiap kali ada perubahan berkas.

Menggunakan Air

Air adalah tool live reload yang dibuat khusus untuk ekosistem Go. Cara memasangnya:

```
go install github.com/air-verse/air@latest
```

Setelah terpasang, jalankan perintah **air** di root direktori proyek. Air akan memantau perubahan berkas **.go** dan otomatis membangun ulang serta menjalankan ulang binary setiap kali ada perubahan yang disimpan.

Menggunakan Nodemon (alternatif)

Bagi yang sudah familiar dengan ekosistem Node.js, **nodemon** juga bisa dikonfigurasi untuk memantau berkas Go dan menjalankan perintah **go run** secara otomatis setiap kali ada perubahan, meski pendekatan ini bersifat opsional dan kurang seefisien Air karena menambah dependency Node.js pada proyek Go.

```
npm install -g nodemon  
nodemon --exec go run main.go --signal SIGTERM
```

Catatan: Buku ini disusun secara orisinal dalam Bahasa Indonesia sebagai materi pembelajaran, mencakup topik-topik standar dalam bahasa pemrograman Go yang umum diajarkan di berbagai sumber belajar. Untuk dokumentasi resmi dan referensi paling mutakhir, silakan kunjungi situs resmi go.dev.